A Low-Cost and Low-Complexity Motion Sensor System Using IMU and Video-Based Tracking

CM2015 HT25-1 Project Carrier Course for Medical Engineers, part 1 15.0 credits

Jack Hutton (hutton@kth.se)
Alexander Jonsson (alejon2@kth.se)
Lisa Kanehl (kanehl@kth.se)
Sahand Khodabandehloo (sahandkh@kth.se)
Lucía Pérez Sáez (luciaps@kth.se)



Kungliga Tekniska högskolan
KTH Royal Institute of Technology

2025-12-19

Abstract

Motion tracking is widely used in healthcare, sports, rehabilitation, and human–computer interaction, yet many established systems remain expensive and require specialized hardware and controlled setups. This project presents a low-cost, low-complexity prototype that combines offline IMU recordings with video-based 2D pose estimation executed entirely in a standard web browser. The application imports IMU CSV data, estimates 2D body keypoints from uploaded videos, and visualizes both modalities side by side with a skeleton overlay and interactive IMU charts. Temporal alignment between video and IMU is achieved via manual event-based synchronization (e.g., clap or jump), where corresponding reference events are marked to estimate a constant time offset. The prototype further supports basic annotation, session persistence, portable export/import, and generation of a plain-text session summary report.

Functional testing on short recordings indicates that the workflow enables practical inspection and documentation of synchronized motion data without dedicated equipment. Limitations include the sensitivity of 2D pose estimation to occlusions and viewpoint, manual synchronization without drift compensation, and the absence of ground-truth validation for numerical accuracy.

Contents

Abstract **2**

1 Introduction **5**
1.1 Motivation . . . 5
1.2 Research objectives . . . 5
1.3 Aim and research question . . . 5
1.4 Requirements . . . 5
1.5 Report structure . . . 5

2 Background **6**
2.1 Inertial Measurement Unit (IMU) . . . 6
2.2 Video pose estimation . . . 6
2.2.1 PoseNet . . . 6
2.2.2 MoveNet . . . 6
2.3 Synchronization . . . 7

3 Methodology **8**
3.1 System overview . . . 8
3.2 Development approach . . . 8
3.2.1 IMU subsystem . . . 8
3.2.2 Video subsystem . . . 8
3.2.3 Integrated browser app for side-by-side IMU-video inspection . . . 9
3.3 Session management . . . 9
3.3.1 Storage, export, import . . . 9
3.3.2 Annotation model: timestamps, events, and notes . . . 11
3.3.3 Report generation . . . 11
3.3.4 Key implementation decisions and limitations . . . 11
3.4 Synchronisation reliability assessment . . . 12
3.5 Summary . . . 12

4 Results **13**
4.1 Test setup and data . . . 13
4.2 Functional results . . . 13
4.3 Synchronisation reliability assessment results . . . 13

5 Discussion **15**
5.1 Feasible but limited functionality . . . 15
5.1.1 Functional validation vs. numerical validation . . . 15
5.1.2 Consistency and stability of outputs . . . 15
5.1.3 Qualitative plausibility of motion data . . . 15
5.1.4 Synchronization reliability outcomes . . . 15
5.1.5 Performance and usability results . . . 16
5.2 Ethical considerations . . . 16
5.2.1 Privacy and data protection . . . 16
5.2.2 Informed consent . . . 16
5.2.3 Misinterpretation and over-reliance . . . 17
5.2.4 Third-party and open-source code . . . 17
5.3 Summary . . . 17

6 Future work **18**
6.1 Multimodal integration and sensor fusion . . . 18
6.2 Pose-estimation benchmarking and robust tracking . . . 18
6.3 Expanded analysis and quantitative validation . . . 18
6.4 Automatic synchronization with a calibration event . . . 18
6.5 Automatic synchronization without explicit events . . . 18
6.6 Additional extensions and scalability . . . 19

7 Conclusion **20**

A Appendix A: Development history of the IMU module **23**

B Appendix B: User guide – web application workflow **24**
B.1 Video upload and skeleton overlay . . . 24
B.2 CSV upload and visualization . . . 24
B.3 Synchronization . . . 25
B.4 Annotation and classification . . . 25
B.5 Import and export . . . 26
B.6 Generate report (TXT summary) . . . 27

1 Introduction

1.1 Motivation

Motion tracking refers to the use of sensing technologies to capture, record, and quantify human movement. Depending on the application, this can be done with camera-based systems, wearable sensors, or a combination of both, producing measurable quantities such as joint positions, velocities, and movement trajectories [11]. By representing movement as numerical data, motion tracking enables systematic analysis beyond subjective visual assessment; for instance, quantitative motion can support the detection of movement deficits, track progress over time, and provide feedback for technique improvement in a reproducible way [1].

Movement measurement is especially relevant in healthcare, sports science, and rehabilitation, where movement quality, efficiency, and consistency directly influence clinical decisions and training outcomes [13][12]. In addition, it can provide additional context for exercises performed in home-based or remote settings [1].

Despite their potential, many established motion tracking solutions remain difficult to deploy in practice. Commercial systems are often expensive, require specialized hardware, and depend on controlled laboratories or studio environments [4]. These barriers limit the availability of objective motion analysis in real-world clinical and training settings.

1.2 Research objectives

The objectives of this project are to (i) acquire and handle IMU recordings, (ii) perform video-based pose (or skeleton tracking) estimation directly in the browser, and (iii) develop a manual synchronization method that aligns the IMU signals with the corresponding video frames despite different time bases, producing time-stamped data streams from both modalities. In addition, the prototype shall provide an interface for side-by-side visualization of synchronized IMU and pose data, supporting basic annotation of motion data.

1.3 Aim and research question

The aim of this project is to design a low-cost, low-complexity, and accessible browser-based motion-tracking prototype that enables practical motion analysis without specialized equipment beyond a camera and an IMU. This leads to the following research question: How can IMU data and video-based pose estimation be integrated into a single browser application for motion analysis?

1.4 Requirements

Firstly, the system should be affordable and not require specialized motion capture equipment. Secondly, it should run in a standard web browser without complex installation. Thirdly, the interface should support an easy workflow for importing IMU recordings, analyzing uploaded videos, visualizing both data sources, and synchronizing them in time. Finally, basic interaction should include video playback with a pose overlay, IMU plots, and simple synchronization markers.

1.5 Report structure

First, the report introduces motivation, objectives, and background on IMUs, pose estimation, and time synchronization. Then, it presents the system design and methods, followed by implementation details of the web application. Finally, the results are demonstrated and discussed, including limitations, ethical considerations, and directions for future work.

2 Background

2.1 Inertial Measurement Unit (IMU)

An Inertial Measurement Unit (IMU) is a sensing device that captures motion using three-axis (x-y-z) sensors: an accelerometer for linear acceleration, a gyroscope for angular velocity, and a magnetometer for orientation relative to the Earth's magnetic field [3]. In this project, Movesense IMUs were used. Although the hardware provides additional functions (e.g., heart-rate features and ruggedized design), this study considered only their motion sensors as well as wireless data transfer and onboard storage for logging measurements.[8] Wearable IMUs are widely used for human motion analysis and rehabilitation because they enable measurement outside laboratory settings.[2]

2.2 Video pose estimation

Video pose estimation is a computer-vision approach that infers a human pose from standard color (RGB) video frames by predicting a set of body keypoints for each frame, which can be visualized as a skeletal overlay [9]. In this project, pose estimation was performed in the browser using TensorFlow.js models. The two detectors considered (PoseNet and MoveNet) are compared in Table 1 [9, 14].

Table 1: Comparison of PoseNet and MoveNet in the context of this project.

Aspect	**PoseNet**	**MoveNet (SinglePose)**
Core output	2D body keypoints (x,y) + confidence per frame	2D body keypoints (x,y) + confidence per frame
Keypoints	17 keypoints (COCO-style)	17 keypoints (COCO-style)
Typical purpose	General-purpose baseline, good for prototyping	efficient inference + stable tracking
Strengths	Well-known, highly configurable; good documentation	Faster and more stable on video; good for browser-based use
Weaknesses	Slower at higher accuracy settings; less stable in fast motion	Fewer classic tuning knobs; mainly choose model variant
Main tunable parameters	`outputStride`, `inputResolution`, `multiplier` (trade speed vs accuracy)	Model variant (Lightning/Thunder), internal tracking/cropping behaviour
Best fit in our app	Baseline / comparison model	Primary model for uploaded-video analysis
Why we chose it	Reference model; exposes classic speed–accuracy trade-offs	Better keypoint reliability for practical video analysis

2.2.1 PoseNet

PoseNet is an established pose-estimation model that was made widely accessible through TensorFlow.js [9]. In this work, PoseNet was primarily used as a baseline to contextualize model choices and to provide a well-documented reference implementation (Table 1) [9].

2.2.2 MoveNet

MoveNet is a newer detector provided via the TensorFlow.js pose-detection API [14]. The model and its surrounding pipeline are designed to improve robustness in video, including measures intended to reduce temporal instability (e.g., jitter) [14]. In this implementation, the detector is instantiated once (e.g.,

`createDetector`) and pose inference is run per frame (e.g., `estimatePoses`), returning 2D keypoints with associated confidence scores [14].

MoveNet offers two single-person variants, Lightning and Thunder. Their trade-offs and the resulting choice for the uploaded-video workflow are summarized in Table 2 [14]. Since the final system focuses exclusively on offline processing of uploaded videos, the selection prioritizes keypoint stability and precision over inference latency.

Table 2: MoveNet variants considered for the uploaded-video workflow.

Aspect	**MoveNet Lightning**	**MoveNet Thunder**
Primary goal	Low latency / speed	Higher accuracy / stability
Typical input size (reported)	192×192	256×256
Keypoint quality (qualitative)	Good, but more jitter/less precise in challenging frames	Better precision and more stable keypoints (esp. occlusions / fast motion)
Runtime (qualitative)	Faster inference	Slower inference
Relevance for uploaded video	Speed is helpful, but not the main constraint	Accuracy/stability is prioritized because processing is offline
Decision in our project	Considered, but not selected	Selected for upload to improve keypoint reliability for analysis

2.3 Synchronization

When combining video-based motion tracking with IMU recordings, both modalities typically follow independent time bases. Synchronization, therefore, refers to establishing a consistent mapping between the video timeline and the IMU timeline such that corresponding samples represent the same physical instant. A common and practical approach is event-based synchronization, where a distinct, clearly identifiable motion event (e.g., a hand clap, jump landing, or sudden knee flexion) is used as a reference point in both signals.

Let t_{Video} denote the timestamp of the reference event in the video and t_{IMU} the corresponding timestamp in the IMU recording. The temporal alignment can then be expressed as a constant time offset $\Delta t = t_{Video} - t_{IMU}$. Using this offset, any IMU sample at time t_{IMU} can be mapped to the video time. This assumption is appropriate when the dominant discrepancy between both data streams is a fixed start-time difference; more complex effects, such as clock drift, would require additional reference points or a more advanced alignment model.

Once synchronized, the IMU signals can be interpreted directly in the context of the observed movement in the video. This improves interpretability by linking quantitative measurements (e.g., accelerations and angular velocities, and potentially derived kinematic variables) to the corresponding visual motion, which supports clearer analysis and reduces ambiguity during inspection and annotation.

3 Methodology

3.1 System overview

The system processes two inputs: a video recording and an IMU recording imported from a CSV file. From the video, it estimates 2D body keypoints and renders them as a skeleton overlay during playback. From the IMU file, it generates timestamped sensor channels and visualizes them as interactive time-series charts. The application also stores user-defined synchronization markers and annotations and supports export/import to save and reload complete analysis sessions. Internally, the workflow consists of a video module (frame decoding and pose estimation), an IMU module (CSV parsing and chart visualization), a synchronization module that aligns both timelines via a constant time offset derived from reference events, and a session module for persistence. The main use case is uploaded-video analysis for reliable inspection of recordings.

3.2 Development approach

The system was developed iteratively. First, two separate browser prototypes were implemented in parallel: one focusing on video-based pose tracking and skeleton visualization, and one focusing on importing and plotting IMU CSV data. After both pipelines were validated independently, they were merged into a single application to enable joint inspection. In subsequent iterations, an event-based synchronization mechanism was added, followed by session export/import to preserve analysis state, and finally an annotation feature to store timestamps, events, and notes within a session.

3.2.1 IMU subsystem

This project uses one Movesense HR2 IMU sensors. Connection with these devices and data extraction was possible with the mobile Movesense Showcase app, however for ease of use and overall simplicity, it was decided to directly connect the sensors to a laptop device through a custom Python application.

For motion analysis, we focus on the tri-axial accelerometer, gyroscope, and magnetometer signals, while other device features are not considered. Recordings are stored and transferred wirelessly and are provided to the web application as CSV files for offline analysis.

In early iterations, a lightweight Python prototype was used to validate Bluetooth connectivity, live streaming, and CSV logging before migrating the processing pipeline to the browser. The code from this app was used as the basis for the initial IMU Sensor WebApp, coded with HTML and JavaScript. Initially, this contained the same features as the Python app, with the addition of a video player, seen in Figure 1.

This WebApp was iterated several times as new features were added, and the project scope shifted. When the IMU data visualization was complete, the code was merged into the Skeleton Tracking webapp to allow for simultaneous video and IMU data viewing.

3.2.2 Video subsystem

The video subsystem performs single-person 2D pose estimation directly in the browser. Video frames are read from an HTML `<video>` element, and the resulting pose is rendered as a synchronized overlay on an HTML `<canvas>`.

Pose estimation is implemented with the TensorFlow.js pose-detection API using MoveNet Thunder. The detector is initialized once, and pose inference is executed by calling `estimatePoses(...)` for selected frames; the function returns 2D keypoints with associated confidence scores.

To maintain usability on consumer devices during uploaded-video analysis, the pipeline applies frame skipping: pose inference is performed only every Nth frame (`UPLOAD_FRAME_SKIP` = 2 in the current configuration), and the most recent pose estimate is reused for intermediate frames. Additionally, uploaded video playback is slowed down slightly (`playbackRate` = 0.75) to reduce computational load while preserving

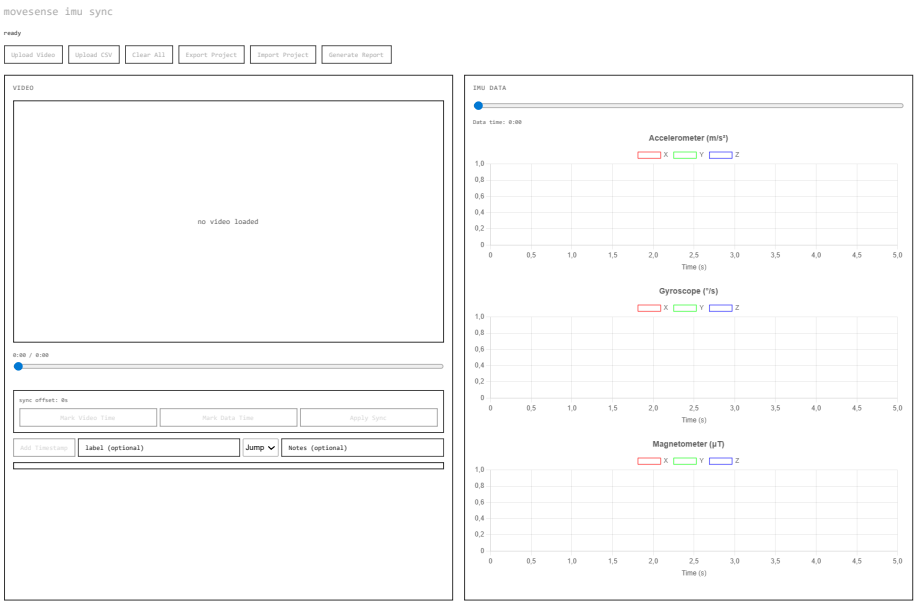


Figure 1: First version of the IMU Video WebApp

a stable overlay. Keypoint visualization is controlled through confidence thresholds: low-confidence keypoints are visually deemphasized, and skeleton connections are omitted if either endpoint falls below the minimum confidence.

3.2.3 Integrated browser app for side-by-side IMU-video inspection

After the IMU visualization and the video-based skeleton tracking were developed in parallel, both were integrated into a single browser application to enable side-by-side inspection of an uploaded video together with the corresponding IMU recording. In upload mode, users load a video and an IMU CSV file; the CSV is parsed into sensor channels with a time axis derived from the fixed sampling rate and displayed as three stacked time-series charts with an interactive "data-time" slider.

Temporal alignment is performed via event-based synchronization. The user marks a reference event in the video and the corresponding event in the IMU view. When "Apply sync" is pressed, the system computes and stores a global syncOffset from the two markers and persists it as part of the project state.

During coupled inspection, the stored offset is applied whenever the video time changes: the IMU view is updated by converting the current video time to the corresponding IMU time (adjustedTime = videoTime − syncOffset) and displaying a sliding time window with a vertical marker at the aligned instant. This update runs both when the video plays without active pose tracking and during tracking. The approach assumes a constant start-time offset between both modalities (no explicit drift model is applied).

3.3 Session management

3.3.1 Storage, export, import

To support reproducible analysis across sessions, the application maintains a per-video project state consisting of the synchronization offset, timestamp annotations, and associated metadata. This state is persisted in the browser via localStorage using a key derived from the active video file (filename and file size), enabling automatic restoration when the same video is loaded again.

For portability and sharing, the project can additionally be exported as a ZIP archive containing a

project.json file (sync offset, timestamps, sampling rate, creation time, and notes) and, if available, the uploaded video and IMU CSV stored under dedicated subfolders. Import supports restoring the full ZIP project by reloading both files and metadata into the application state; alternatively, a metadata-only JSON import is supported, in which case video and CSV must be provided separately. The following figures show the workflow for saving video and CSV files.

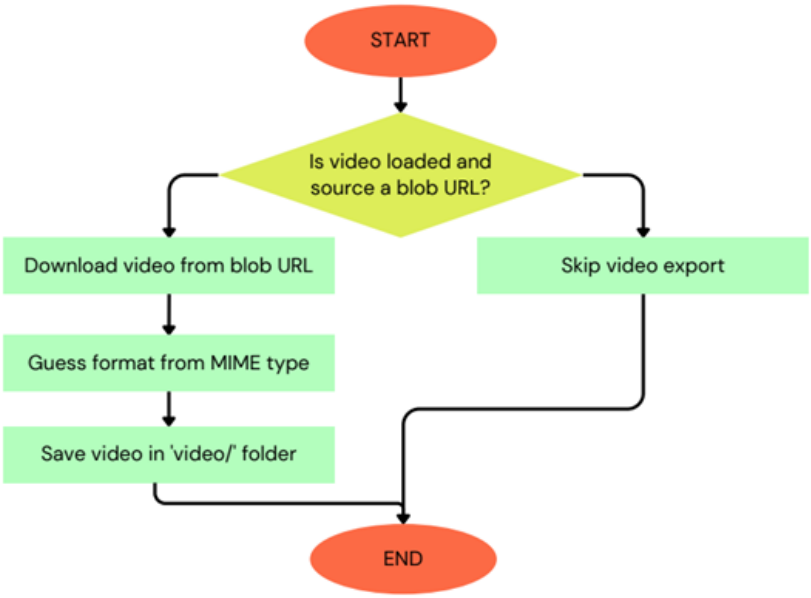


Figure 2: Workflow for exporting the video file to the video/ folder

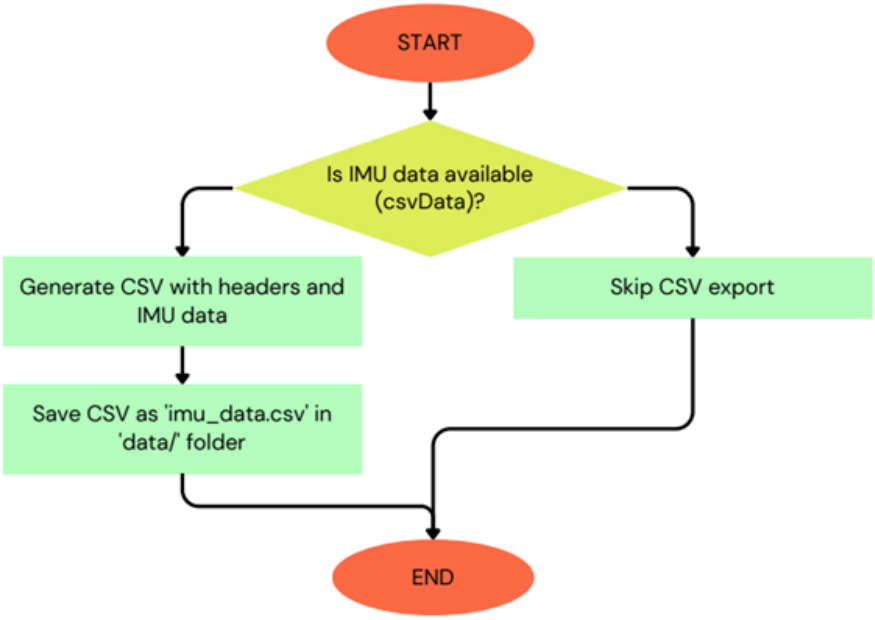


Figure 3: Workflow for exporting the IMU CSV file to the data/ folder.

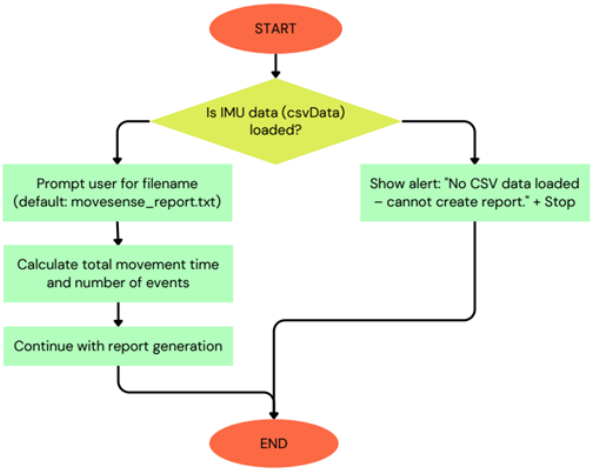


Figure 4: Workflow for creating the IMU report .txt file from the loaded CSV.

3.3.2 Annotation model: timestamps, events, and notes

Annotations are represented as an ordered list of timestamp objects with the fields time, label, eventType, notes. The stored time value refers to the video timeline, which ensures that annotations remain stable with respect to the visual reference and can be mapped to IMU time through the synchronization offset. The event type is selected from a small, predefined set (Clap/Jump/Other), and optional free-text notes can be attached to each entry. Selecting an annotation seeks the video to the stored time and updates the IMU view accordingly, allowing qualitative inspection of the corresponding motion together with the quantitative sensor signals.

3.3.3 Report generation

A lightweight reporting function serialises the current project state into a human-readable TXT file. The total recording duration is derived from the IMU time axis (using the last value of csvTimesSeconds), and the number of stored annotations is taken from the current timestamps list.

Each annotation is written as an indexed entry containing the label, the stored event time (kept in video time via video.currentTime), the event type, and optional free-text notes. The report text is then generated fully client-side by creating a UTF-8 Blob and triggering a download through a temporary object URL, avoiding any server-side processing.

3.3.4 Key implementation decisions and limitations

Several implementation choices were made to maintain usability on consumer hardware. For uploaded video analysis, pose inference is executed with frame skipping (`UPLOAD_FRAME_SKIP` = 2) and playback is slightly slowed down (`playbackRate` = 0.75) to reduce compute load while preserving a stable overlay. Keypoint rendering is filtered using a minimum confidence threshold (`MIN_PART_CONFIDENCE` = 0.2).

On the IMU side, timestamps are derived from a fixed sampling rate (`SAMPLE_RATE_HZ` = 104 Hz). We used a sampling rate of 104 Hz because the Movesense HR2 accelerometer and gyroscope support discrete sampling frequencies including 12.5/26/52/104/208/416/833 Hz.[7] The application assumes a constant sampling rate to convert sample indices into time for synchronized playback with the uploaded video. Chart updates are throttled and displayed within a moving time window around the currently aligned time to avoid excessive redraw overhead during playback.

The main limitations are the use of 2D pose keypoints (sensitive to occlusion and camera viewpoint), per-

formance variability across devices, and the synchronization model assuming a constant time offset without drift compensation. In addition, IMU timing accuracy depends on the correctness of the assumed sampling rate and stable logging behavior.

3.4 Synchronisation reliability assessment

To investigate the current manual synchronisation approach a simple reliability test was designed and conducted. Using the event-based approach and a clap action, three sets of csv data with the corresponding videos were created (trials). All of these featured synchronization events (a clap) with offsets between 1.5 and 3.5 seconds. Five raters manually synchronised all three sets using the web application. Inter-rater reliability was assessed using an intraclass correlation coefficient, and descriptive statistics and visualisations were used to compare time offsets.

3.5 Summary

This section described the methods used to build and operate the browser-based IMU–video motion analysis prototype. It outlined the IMU processing pipeline (CSV import, timestamp reconstruction, and chart visualization) and the video pose-estimation pipeline (MoveNet for uploaded videos with an optional live mode). It also introduced the event-based synchronization approach, together with session persistence (local storage and export/import) and the annotation and report-generation features. The following section presents the resulting prototype behaviour and evaluates its functionality, usability, and practical limitations.

4 Results

4.1 Test setup and data

To evaluate the prototype, we carried out functional tests primarily in the upload workflow, using short prerecorded videos together with recorded IMU CSV data. The uploaded test videos were limited to a maximum duration of 20 s and contained simple, clearly identifiable movements such as claps and short dynamic actions. During playback, the application processed video frames and rendered the estimated 2D skeleton overlay on a canvas aligned with the video element.

IMU data was collected with a single Movesense HR2 sensor held in the hand and exported as CSV files containing tri-axial accelerometer, gyroscope, and magnetometer channels. The IMU time axis was reconstructed from the sample index using a fixed sampling rate of 104 Hz. This rate is supported by the Movesense HR2 motion sensors and is used by the application for converting sample indices into timestamps during synchronized playback with the uploaded video [7].

Synchronization between modalities was performed manually by selecting a distinctive reference event visible in both the video and the IMU signals. All tests were executed locally in a standard web browser on a consumer laptop. The prototype runs fully client-side in the browser, using TensorFlow.js for pose estimation and Chart.js for IMU visualization.

4.2 Functional results

The prototype successfully supports uploaded-video playback with a 2D skeleton overlay, using MoveNet Thunder as the detector for this mode. To keep the overlay responsive on consumer hardware, the implementation applies performance optimizations (pose updates on a subset of frames and a reduced playback rate), which stabilized rendering during our tests. IMU recordings can be imported from CSV files, converted into a time axis, and visualized as three stacked charts (accelerometer, gyroscope, magnetometer) with a data-time slider for navigation.

Manual event markers produce a stored syncOffset, and after applying synchronization, the IMU view follows the current video time during playback and scrubbing by updating a sliding chart window and aligning the marker. Project state (including syncOffset and stored timestamp annotations) can be exported as a ZIP project containing project.json and re-imported to restore both data and metadata. Users can create timestamp annotations with labels, event types, and notes, and a TXT summary report can be generated from the loaded IMU data and the stored event list.

4.3 Synchronisation reliability assessment results

The manually applied offsets from the synchronisation reliability assessment for all raters are summarised in Table 3. These offsets were plotted against others from the same trial for visual comparison, seen in Figure 5.

Table 3: Applied manual synchronisation offsets for each trial and rater. Offsets are reported in seconds.

Trial	Rater 1	Rater 2	Rater 3	Rater 4	Rater 5
Trial 1	-3.215	-3.355	-3.300	-3.298	-3.317
Trial 2	-1.991	-1.844	-1.982	-1.939	-1.968
Trial 3	-1.949	-1.721	-2.008	-1.970	-1.943

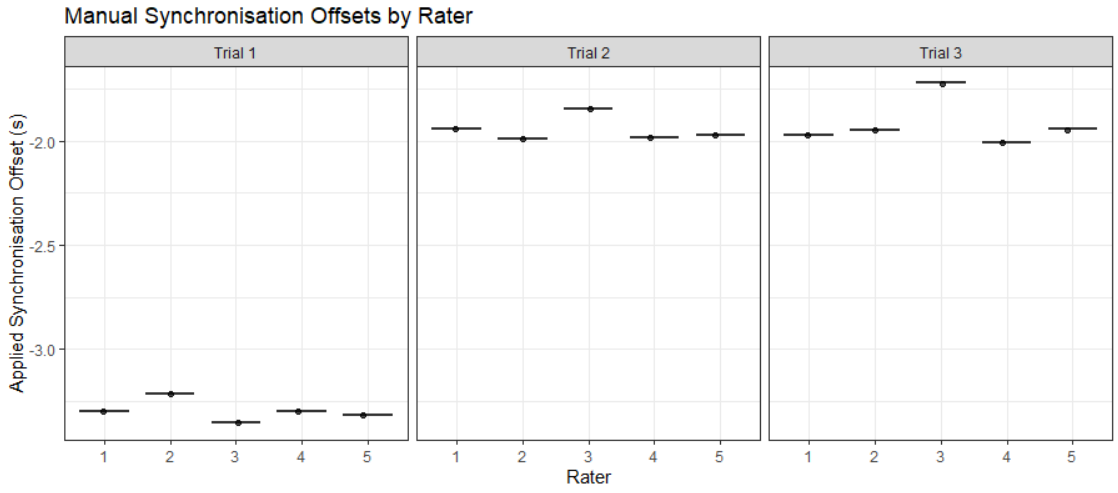


Figure 5: Manual Synchronisation Offsets by Rater.

Inter-rater reliability was assessed using the intraclass correlation coefficient (ICC; two-way random-effects model, absolute agreement). The ICC was 0.989, with 95% confidence intervals from 0.945 and 1, indicating excellent inter-rater agreement despite the small number of trials.

To complement the ICC value, the mean absolute deviation of the synchronisation offsets from the trial mean was calculated, seen in Table 4. The small deviations observed across trials indicate that the raters were generally consistent in applying the synchronisation offset.

Table 4: Mean absolute deviation of synchronisation offsets from the trial mean for each dataset.

Trial	Mean Absolute Deviation (s)
1	0.0406
2	0.0458
3	0.0874

5 Discussion

The results indicate that a low-cost, browser-based workflow can integrate video pose estimation with IMU recordings and support basic inspection, synchronization, and annotation in a single interface. The upload-focused design is particularly suitable for the project goals, because offline processing reduces dependence on real-time compute performance and improves stability compared to a purely live system. Overall, the prototype is best interpreted as an exploratory analysis tool rather than a validated measurement system for professional use.

5.1 Feasible but limited functionality

The prototype demonstrates that IMU recordings and video-based pose estimates can be combined in a lightweight browser workflow, but the current scope remains intentionally limited. In its present form, the system supports comparative inspection of two modalities rather than performing true sensor fusion or producing calibrated, physically validated kinematic quantities. Consequently, the outputs are most appropriate for exploratory analysis and documentation, while professional deployment would require stronger calibration, robustness measures, and validation.

5.1.1 Functional validation vs. numerical validation

The evaluation primarily addresses whether the workflow behaves consistently and supports the intended analysis steps, but it does not provide a rigorous assessment of measurement accuracy. Without ground-truth reference data, it is not possible to quantify systematic errors or precision in absolute terms, and comparisons to high-end motion-capture systems remain conceptual. In addition, the pose component is inherently constrained by 2D, single-person keypoint estimation, which limits the range of motions and scenarios for which strong conclusions can be drawn.

5.1.2 Consistency and stability of outputs

From a methodological perspective, stability depends on both algorithmic choices and available computational resources. Any browser-based, client-side implementation must balance responsiveness against model complexity, and performance can degrade on less capable devices or in demanding scenes. Measures that increase stability (e.g., reducing inference frequency) can introduce trade-offs such as lower temporal resolution, which is relevant when analysing fast or short-duration events.

5.1.3 Qualitative plausibility of motion data

Qualitative inspection suggests that the produced trajectories and sensor patterns are interpretable, but visual plausibility is not equivalent to quantitative correctness. Pose estimation quality can vary with viewpoint, occlusion, and lighting, and IMU signals can be affected by placement and magnetic interference. As a result, the most defensible interpretation is that the prototype provides a coherent visual-and-signal context for motion events, while numerical interpretation should be treated cautiously.

5.1.4 Synchronization reliability outcomes

The manual, event-based alignment method improves interpretability by connecting visible events to sensor peaks, but its accuracy is bounded by the user's event selection and by the assumptions of the alignment model. In particular, a constant-offset approach does not account for clock drift or variable latency, which can accumulate error over time. More robust alignment would require additional reference events or a drift-aware model.

The reliability analysis performed is based on a limited number of datasets (three trials), which limits the statistical power of the estimated reliability metrics. While the observed ICC (0.989) indicates high

inter-rater reliability, such values can be inflated in small sample sizes and should be interpreted cautiously, as seen in the wide confidence intervals. The mean absolute deviation (Table 4) provides a complementary absolute measure of rater spread. This remained low across the three trials despite variation in the absolute offset (Table 3). Combined, these measures indicate that the current manual event-based alignment approach is reliable and consistent, but future consideration should be given to an automated system.

From a system perspective, the prototype's responsive video playback with a 2D skeleton overlay and the linked IMU view during playback demonstrate that manual alignment is feasible on consumer hardware, which supports the usability of the synchronization interface. In addition, the low mean absolute deviations from Table 4 (0.0406–0.0874 s) suggest that raters typically selected near-identical alignment points. Nevertheless, the slightly higher deviation in Trial 3 reflects the fact that the current system relies on manual alignment, which makes event identification more ambiguous and increases user-dependent uncertainty. Furthermore, the performance optimizations (pose estimation on a subset of frames and reduced playback rate) may influence perceived event timing and should be considered when interpreting synchronization precision. The ability to export/import project state (including `syncOffset` and annotations) strengthens reproducibility, but validating alignment accuracy against a ground-truth reference remains an important next step, alongside exploring automated or semi-automated synchronization methods.

5.1.5 Performance and usability results

The browser-based design lowers deployment barriers and supports rapid iteration, which is valuable for prototyping and low-cost settings. However, usability is coupled to performance: responsiveness, smooth playback, and stable overlays are all influenced by device capability and environmental conditions. For broader applicability, future work should therefore treat performance and validation as first-class goals, alongside the existing emphasis on simplicity and accessibility.

5.2 Ethical considerations

An ethical framework is essential for ensuring user safety and public trust. Since this project involves the collection and processing of human motion data derived from video recordings and IMU sensors, it raises several ethical considerations related to privacy, informed consent (respect for autonomy), data handling, and potential misuse.

5.2.1 Privacy and data protection

First, privacy and data protection are central concerns, particularly because video data may contain identifiable information about individuals. To mitigate the risk, the system is designed to operate locally in the user's web browser, meaning no video recordings or motion data are transmitted to or processed on external servers; processing is performed locally in the user's browser. Project metadata may be stored locally in the browser and can optionally be exported/imported by the user. Thus, users retain full control over their data from start to finish. This design choice aligns with principles of data minimization and reduces the risk of unauthorized access or data breaches, which further ensures public trust.

5.2.2 Informed consent

Second, informed consent, or respect for autonomy, from the bioethical framework of principlism — i.e., the right of an individual to make a self-determined choice, given that they are fully informed in the context at hand — is essential when other people, namely the users, are involved. In this project, all testing was conducted voluntarily and for educational and prototype purposes only. The testing of the system was conducted by the group members — not external participants. For any future advancements or deployment in clinical, rehabilitation, or sports contexts, explicit consent procedures and clear user information would be required.

5.2.3 Misinterpretation and over-reliance

Third, there is a risk of misinterpretation or over-reliance on system output. Due to the limited accuracy and lack of clinical validation — since the motion data was only extracted from the IMUs, not compared or validated with other data — the system should not be used for diagnosis or treatment decisions, especially in medical or rehabilitation settings. Instead, it should be used as a supportive or exploratory tool. Consequently, the project raises questions about equity and accessibility. While the low-cost and browser-based design improves accessibility and makes it less hard to operate compared to high-end motion capture systems, it may also lead to use by non-expert users without sufficient understanding of its limitations. Therefore, clear disclaimers and usage guidelines would be necessary in any real-world application to ensure ethical consistency.

5.2.4 Third-party and open-source code

Finally, ethical concerns due to the use of third-party and open-source code are relevant. A significant portion of the underlying code and system functionality is based on existing implementations made publicly available by other developers in open-source platforms (e.g., GitHub) and online technical resources. This approach was intentionally chosen, given a recommendation by the project supervisor, as it reflects common practice in modern software engineering and enables efficient development within the constraints of a course project. All reused code was adapted, integrated, and understood within the context of the system. In line with this approach, the project also makes the developed source code publicly available in GitHub, enabling transparency, reproducibility, and further development by others. This allows future students, researchers, or developers to reuse, evaluate, and improve the system, thereby contributing back to the same open-source ecosystem on which this project was built.

5.3 Summary

In summary, the discussion suggests that the proposed prototype is not intended to replace high-end motion capture systems but to provide a low-cost and accessible alternative for exploratory motion analysis in non-laboratory settings. The identified limitations and trade-offs outline clear directions for future work and reflect practical constraints that influence real-world deployment.

6 Future work

6.1 Multimodal integration and sensor fusion

Several enhancements could be considered to extend the system's capabilities and improve robustness. A first direction is the tighter integration of both modalities. Integrating sensor-fusion methods (e.g., complementary filtering or Kalman filtering) could combine IMU-derived motion cues with video keypoints and support more stable motion interpretation [6]. For example, IMU orientation estimates could help stabilize joint trajectories when the 2D keypoints temporarily degrade due to occlusions or fast motion, while video keypoints could provide additional spatial context that is difficult to infer from IMUs alone.

6.2 Pose-estimation benchmarking and robust tracking

Motion tracking performance could also be improved through systematic benchmarking and parameter tuning. This includes comparing pose-estimation settings such as MoveNet model choice, frame-skipping rate, and confidence thresholds, and evaluating their effect on keypoint stability and runtime performance [14, 9]. In addition, robustness in real-world recordings could be improved by explicitly handling challenging conditions (e.g., partial occlusions, motion blur, and variable lighting), for instance by applying temporal smoothing, confidence-weighted rendering, or automatically warning the user when tracking quality is low.

6.3 Expanded analysis and quantitative validation

A further extension concerns evaluation and analysis functionality. Adding more built-in analysis tools (e.g., joint-angle estimates, simple range-of-motion summaries, or event-based statistics) would enable more detailed post-processing directly within the prototype. Moreover, controlled experiments against reference measurements (e.g., known timing events or external motion capture for selected trials) would enable quantitative validation of accuracy and repeatability, which is important for interpreting the results in biomechanics and rehabilitation contexts [2, 4].

6.4 Automatic synchronization with a calibration event

A particularly relevant extension concerns synchronization. The current workflow relies on manual event marking and assumes a constant time offset. Future versions could reduce user effort and improve repeatability by introducing an automatic alignment approach [10]. One practical approach is using a predefined calibration event (e.g., a clap as in our current manual approach) at the beginning of the recording. On the video side, the clap can be detected using hand or wrist keypoints (e.g., when the distance between both hands becomes very small), while on the IMU side, the same event can be detected by a distinct peak in acceleration magnitude [10]. The first matched event in both modalities can then be used to compute the offset automatically and to initialize synchronized playback.

6.5 Automatic synchronization without explicit events

As an alternative, synchronization could be estimated without an explicit calibration event by comparing overall movement intensity over time and selecting the time shift that matches both signals best. Concretely, a video-derived motion-intensity signal can be computed from keypoint motion between frames, and an IMU-derived intensity signal can be obtained from the acceleration magnitude. The temporal offset can then be estimated by maximizing the cross-correlation between these two intensity signals [10]. This approach can exploit longer recording segments and may be useful when no single clean calibration event is available. Beyond a single constant offset, future work could also consider drift-aware alignment models, for example, by estimating offsets at multiple points in time and fitting a simple correction model, or by using online estimation of the camera–IMU temporal offset as described in the literature [5].

6.6 Additional extensions and scalability

Further extensions include tracking multiple people in uploaded videos, importing multiple sensor streams, and supporting synchronized real-time IMU input for more integrated motion capture workflows. These additions would expand the range of use cases, such as group exercises, multi-segment body instrumentation, and live feedback during rehabilitation sessions. However, they also increase system complexity and computational demands, which would require careful UI design and performance optimization to maintain stable real-time interaction.

7 Conclusion

This project demonstrated the feasibility of a low-cost, lightweight motion analysis prototype that runs entirely in a standard web browser. The system integrates uploaded video-based 2D pose estimation with imported Movesense IMU recordings, providing synchronized side-by-side visualization, manual event-based alignment using a constant time offset, and basic annotation of motion events. In addition, the prototype supports session persistence via local storage, portable export/import of project data, and automatic generation of a plain-text session summary from the IMU timeline and the stored annotations.

The prototype is intended for exploratory inspection and documentation rather than as a substitute for laboratory-grade motion capture. Its main limitations are the reliance on 2D keypoints (susceptible to occlusions and camera viewpoint), manual synchronization without drift compensation, and the lack of ground-truth validation of numerical accuracy. Future work should therefore focus on quantitative benchmarking of pose stability and timing alignment, drift-aware or automated synchronization, and tighter multi-modal integration (e.g., fusion of IMU orientation estimates with video keypoints) to improve robustness and measurement reliability.

References

[1] Afrooz Arzehgar, Seyedeh Nahid Seyedhasani, Fatemeh Baharvand Ahmadi, Fatemeh Bagheri Baravati, Alireza Sadeghi Hesar, Amir Reza Kachooei, and Shokoufeh Aalaei. Sensor-based technologies for motion analysis in sports injuries: a scoping review. *BMC Sports Science, Medicine and Rehabilitation*, 17(1):15, January 2025. Published 30 January 2025.

[2] Andrea Cereatti, Reed Gurchiek, Annegret Mündermann, Silvia Fantozzi, Fay Horak, Scott Delp, and Kamiar Aminian. Isb recommendations on the definition, estimation, and reporting of joint kinematics in human motion analysis applications using wearable inertial measurement technology. *Journal of Biomechanics*, 173:112225, 2024.

[3] Conoptics. Inertial measurement unit: Essential guide to imu technology. Online article, January 2025. Accessed 2026-01-10.

[4] Jinfeng Li, Fanji Qiu, Liaoyan Gan, and Li-Shan Chou. Concurrent validity of inertial measurement units in range of motion measurements of upper extremity: A systematic review and meta-analysis. *Wearable Technologies*, 5:e11, October 2024. Published online 04 October 2024.

[5] Mingyang Li and Anastasios I. Mourikis. Online temporal calibration for camera–imu systems: Theory and algorithms. *The International Journal of Robotics Research*, 2014.

[6] Sebastian O. H. Madgwick. An efficient orientation filter for inertial and inertial/magnetic sensor arrays. Technical report, University of Bristol, April 2010. Technical Report.

[7] Movesense. *Movesense Sensor HR2: Spec Sheet / Technical Brief*, August 2021. PDF, version 08/2021.

[8] Movesense. Technical specifications. Web page, n.d. Accessed 2026-01-10.

[9] Dan Oved. Real-time human pose estimation in the browser with tensorflow.js. TensorFlow Blog, May 2018. Posted May 7, 2018; accessed 2026-01-10.

[10] Thomas Plötz, Chen Chen, Nils Y. Hammerla, and Gregory D. Abowd. Automatic synchronization of wearable sensors and video-cameras for ground truth annotation – a practical approach. In *Proceedings of the 16th International Symposium on Wearable Computers (ISWC)*, pages 100–103, 2012.

[11] Johanna E. van Schaik and Nadia Dominici. Motion tracking in developmental research: Methods, considerations, and applications. In Sabine Hunnius and Marlene Meyer, editors, *New Perspectives on Early Social-cognitive Development*, volume 254 of *Progress in Brain Research*, pages 89–111. Elsevier, 2020.

[12] Verena Venek, Stefan Kranzinger, Hermann Schwameder, and Thomas Stöggl. Human movement quality assessment using sensor technologies in recreational and professional sports: A scoping review. *Sensors*, 22(13):4786, 2022.

[13] Claire Villepinte, Arpana Verma, Chloe Dimeglio, Xavier De Boissezon, and David Gasq. Responsiveness of kinematic and clinical measures of upper-limb motor function after stroke: A systematic review and meta-analysis. *Annals of Physical and Rehabilitation Medicine*, 64(2):101366, March 2021.

[14] Ronny Votel and Na Li. Next-generation pose detection with movenet and tensorflow.js. TensorFlow Blog (Google Research), May 2021. Posted May 17, 2021; accessed 2026-01-10.

Team contributions

At the start of the project, the team divided into two subgroups: a video-focused group and an IMU-focused group. Sahand Khodabandehloo, Alexander Jonsson, and Lisa Kanehl were responsible for establishing the initial web-based video pipeline, including video upload and browser-based skeletal tracking. In parallel, Jack Hutton and Lucía Pérez Sáez focused on the IMU subsystem, including importing and visualizing IMU CSV recordings, implementing multi-axis plotting, and investigating sensor interaction and data handling.

After the initial subsystems were established, the team merged efforts to build and refine a single integrated website that supports uploaded video analysis together with synchronized IMU visualization. The implementation was iteratively improved through regular coordination meetings, during which remaining tasks were identified and distributed. Throughout the project, progress and next steps were discussed in recurring supervision meetings with Jonas Willén.

Overall, contributions were balanced across the group. Team members supported each other across components as needed, and effort was distributed across implementation, integration, testing, documentation, and presentation materials (report and poster), ensuring that all members contributed comparably to the final outcome.

A Appendix A: Development history of the IMU module

To reduce implementation risk, the IMU pipeline was validated in several iterations before being integrated into the final browser-based system. Initial connectivity and data acquisition were tested using a lightweight Bluetooth Low Energy (BLE) prototype based on an existing template (using the bleak library) to confirm stable pairing, streaming, and access to the relevant sensor signals. As illustrated in the following figure, the initial Bluetooth application is displayed.

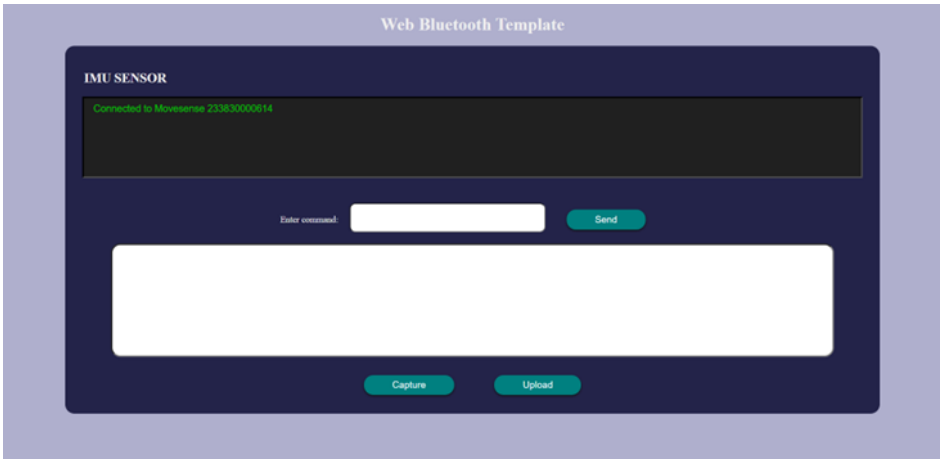


Figure 6: Initial BLE-based IMU web application prototype

After basic connectivity was verified, a small Python application (seen in the following figure) was developed to visualize live accelerometer, gyroscope, and magnetometer data and to support recording and saving measurements as CSV files. This step served as a functional prototype to validate the end-to-end IMU workflow (acquisition → logging → reloading → plotting) independently of the web interface.

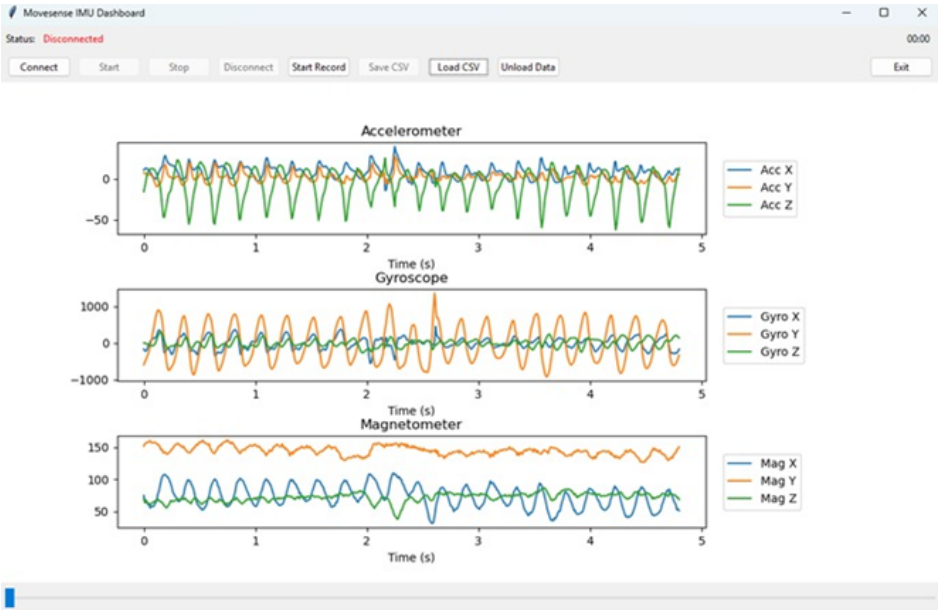


Figure 7: Python□based BLE IMU dashboard used to visualize live sensor data and record CSV logs.

B Appendix B: User guide – web application workflow

B.1 Video upload and skeleton overlay

When opening the web application, the first step is to load a video file. Click the “Select video” button, and a pop-up will appear, allowing you to choose a pre-recorded file from your device’s file explorer.

Once the video is loaded, the user must click ”Start” to activate skeleton tracking. A slider above the video enables navigation through the recording. When ”Start” is active, the skeleton tracking continues to run as the user moves the slider; when ”Pause” is pressed, the tracking is paused. All changes to the project should be made while the ”Pause” button is active.

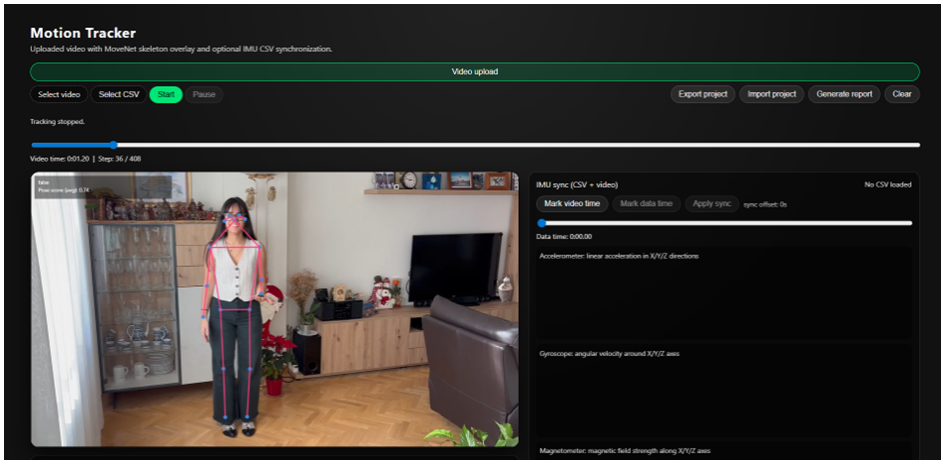


Figure 8: Motion tracker interface for video upload and skeleton overlay.

B.2 CSV upload and visualization

After uploading the video, the user proceeds to upload the IMU data by clicking the ”Select CSV” button. A pop-up window appears, prompting the user to select the .csv file from the device’s file explorer. Once uploaded, the user can navigate through the sensor data using a slider. The application displays three real-time graphs for the accelerometer, gyroscope, and magnetometer data, as previously described.

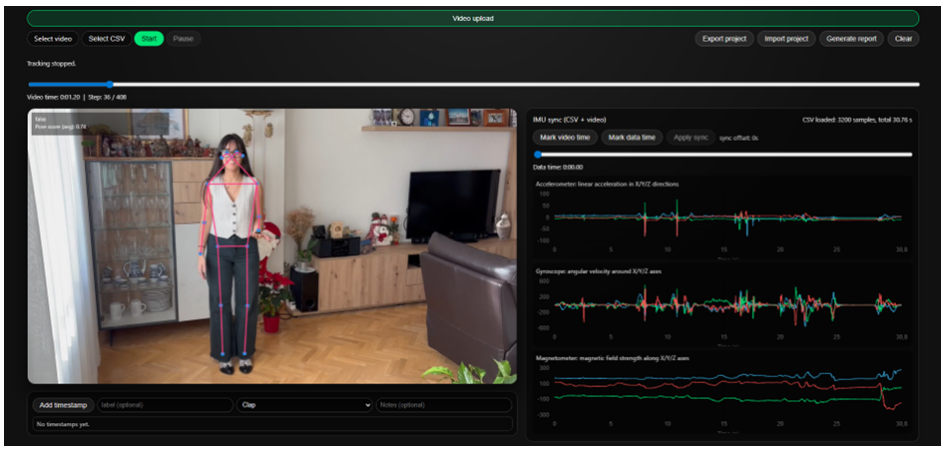


Figure 9: Motion tracker interface for IMU CSV upload and signal visualization

B.3 Synchronization

To align both modalities, use the synchronization controls after both files are loaded and while the Pause button is selected. Navigate to a distinctive motion event in the video (e.g., a clap or jump landing) and mark the time using the “Mark video time” button. Then navigate to the corresponding event in the IMU plots and mark it using the “Mark data time” button. Click “Apply sync” to align the time axes so that video playback and IMU inspection refer to the same physical moment.

From then on, the slider above the video controls both the video and the graphs in a synchronized way.

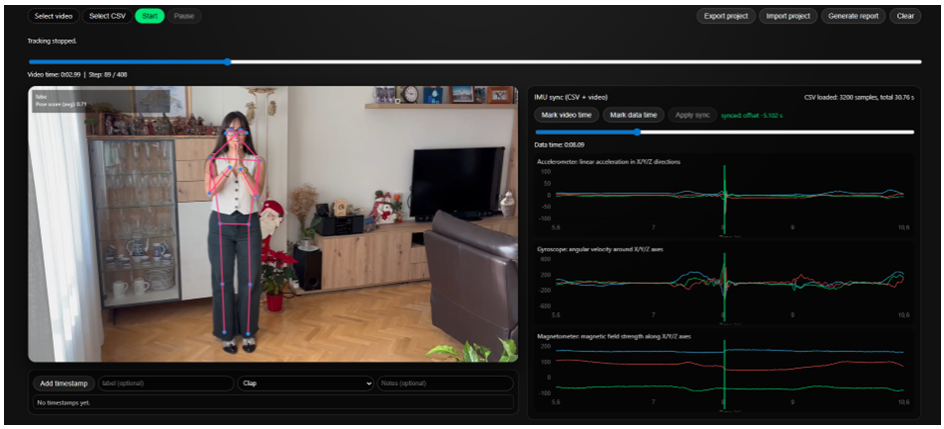


Figure 10: Manual video–IMU synchronization using the mark time controls

B.4 Annotation and classification

After synchronization, relevant moments can be annotated. Navigate to the desired video time and enter:

- Label (a short name for the event)
- Classifier (prototype categories: Jump, Clap, Other)
- Note (optional free-text description)
- Press Add Timestamp to store the annotation. Stored timestamps can be used to document events of interest and support later review.

For example, if the user wants to highlight a moment where the subject is jumping, they can navigate to that frame, enter a label (e.g., ”First Jump”), and classify the action using the dropdown menu (e.g., ”Jump,” ”Clap,” ”Other”). The note input box allows the user to add descriptive text, such as ”Lucia’s first jump during training session number 13, first try of this new exercise, analyzing if her performance has improved compared to previous sessions.” After completing these fields, the user clicks ”Add Timestamp” to store the annotation.

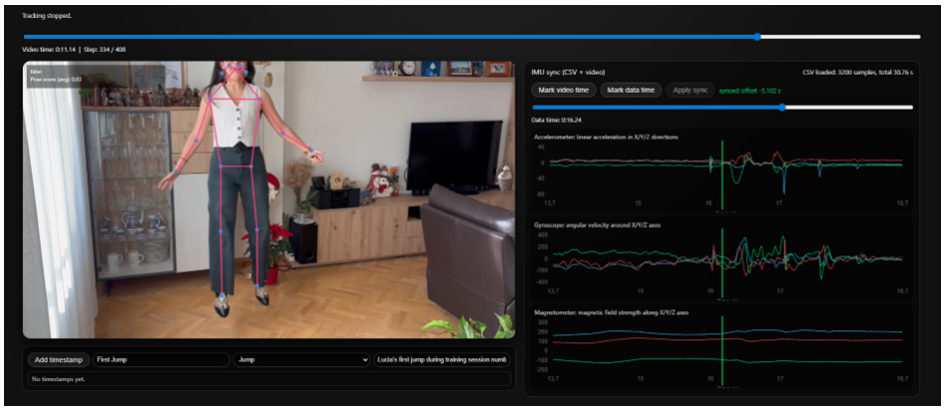


Figure 11: Enter Caption

Once the timestamp is stored, it will appear in a list of timestamps below. The user can navigate through them, and when one is selected, the view will jump to that moment and display the stored information.

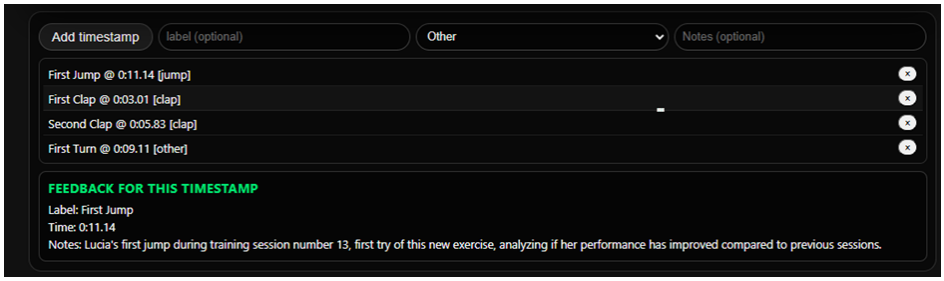


Figure 12: Timestamp list and detailed feedback view for a selected event

B.5 Import and export

Currently, the project data is managed in the browser using LocalStorage, which stores only metadata (not the actual video or .csv). When you upload a video and/or a .csv file and change anything—apply a time sync, add timestamps, labels, or notes—that information is saved in LocalStorage and linked to the names of the video and .csv files. If you clear the page or close the browser, this metadata is removed, but when you upload the same video and .csv again, all your changes are restored: the video and IMU stay synchronized and your timestamps and notes appear again.

To make a project reusable, portable, and shareable between users (researchers, patients, colleagues) and to avoid losing work when changing computers or having system issues, use the "Import Project" and "Export Project" buttons.

Export Project

When you click "Export Project" a pop-up asks you for a filename for the ZIP. The app creates a ZIP with this structure, containing your data:

```
movesense_project.zip
 project.json
 video/
    video_from_project.mp4
 data/
    imu_data.csv
```

The file project.json stores your session metadata:

- `syncOffset`: synchronization offset between video and IMU.
- `timestamps`: list of timestamps and events.
- `sampleRate`: IMU sampling rate.
- `createdAt`: date and time when the project was created.
- `notes`: any additional notes.
- `videoFileName`: generic video filename (for example, `video_from_project.mp4`).
- `csvFileName`: generic CSV filename (for example, `imu_data.csv`).

Import Project

When you want to import a project, click the "Import Project" button and select a ZIP that was exported previously. The app reads the ZIP, loads the video and the .csv, and applies all metadata from project.json. The project is restored exactly as before: video and IMU are synchronized, timestamps, labels, and notes are back, and you can continue working without repeating setup steps.

B.6 Generate report (TXT summary)

To generate a simple summary, use Generate Report. The application produces a plain-text report that includes the total recording duration, the number of stored events, and a list of all annotations (index, label, timestamp, event type, and optional notes). The TXT file is downloaded automatically and can be used for documentation or sharing results.